STREAMLIT

Complete Beginner's Guide

Building Machine Learning Projects with Python

Artificial Intelligence

BS (CS) Spring 2026

1. Introduction to Streamlit

Streamlit is an open-source Python framework that allows data scientists and ML engineers to build and share beautiful, interactive web applications — without writing any HTML, CSS, or JavaScript. In just a few lines of Python code, you can turn your machine learning model into a polished, shareable web app.

Why Streamlit?

Traditional web development for ML projects requires learning Flask or Django, HTML/CSS for styling, JavaScript for interactivity, and REST API design. Streamlit eliminates all of that. Here is a comparison:

Without Streamlit	With Streamlit
Learn Flask/Django + HTML/CSS/JS	Just write Python — no extras needed
Days or weeks to build a UI	Minutes to build a functional UI
Separate frontend and backend code	Everything in one Python script
Requires web dev expertise	Beginner-friendly for data scientists
Complex deployment setup	One-command deploy with Streamlit Cloud

Key Features

- Instant reloading — app updates as you code
- Rich widgets: sliders, dropdowns, file uploaders, buttons
- Built-in support for charts: Matplotlib, Plotly, Altair
- Caching system for fast ML model loading
- Easy deployment via Streamlit Community Cloud
- Sidebar, columns, tabs, and expanders for layout

> Streamlit was founded in 2018 and acquired by Snowflake in 2022. It has over 1 million users worldwide and is one of the most popular tools for building ML demos and dashboards.

1.1 What You Will Build in This Guide

By the end of this guide, you will have built a complete Machine Learning web application that:

- Accepts user input through interactive widgets
- Loads a trained ML model and runs predictions in real time
- Displays results with charts, tables, and visual feedback
- Is ready to deploy and share publicly on the internet

1.2 Prerequisites

Before starting, you should have:

- Python 3.8 or higher installed on your machine
- Basic familiarity with Python syntax (variables, functions, loops)
- pip (Python package installer) available in your terminal
- A code editor such as VS Code, PyCharm, or even a text editor

> **No ML experience required!**
> This guide is structured so that even if you have never trained a machine learning model before, you can still follow along. We will use a pre-built dataset and a simple model to keep the focus on Streamlit.

2. Installation & Setup

2.1 Setting Up Your Environment

It is best practice to use a virtual environment for every Python project. This keeps your dependencies isolated and prevents conflicts between projects.

Step 1 — Create a project folder

```
# Create a new directory for your project
mkdir my-streamlit-ml-app
cd my-streamlit-ml-app
```

Step 2 — Create a virtual environment

```
# Windows
python -m venv venv
venv\Scripts\activate

# macOS / Linux
python3 -m venv venv
source venv/bin/activate
```

Step 3 — Install Streamlit and ML libraries

```
pip install streamlit scikit-learn pandas numpy matplotlib plotly
```

Step 4 — Verify installation

```
streamlit --version
# Expected output: Streamlit, version 1.x.x
```

2.2 Your First Streamlit App

Create a file called app.py in your project folder and add the following code:

```
import streamlit as st

st.title('My First Streamlit App!')
st.write('Hello, World!')

name = st.text_input('Enter your name:')
if name:
    st.success(f'Hello, {name}! Welcome to Streamlit.')
```

Run it with:

```
streamlit run app.py
```

Your browser will automatically open at http://localhost:8501 and display your app. Every time you save the file, Streamlit will reload the app automatically.

> **Hot Reloading**
> Streamlit watches your file for changes. You can keep the terminal open and just edit and save your app.py file — the browser will refresh automatically. No need to restart the server.

3. Core Streamlit Concepts

Before building the full ML app, it is important to understand how Streamlit works under the hood and the building blocks it provides.

3.1 How Streamlit Works

Streamlit re-runs your entire Python script from top to bottom every time a user interacts with a widget. This is different from traditional web apps where only part of the page updates. This model is simple but powerful — you always write your app as if it is running for the first time.

> Streamlit execution model: Every user interaction triggers a full script re-run. Streamlit is smart enough to only re-render parts of the UI that actually changed, making it feel fast and responsive.

3.2 Display Elements

Streamlit provides many functions to display text, data, and media:

Function	Description	Example
st.title()	Large title text	st.title('My App')
st.header()	Section header	st.header('Results')
st.subheader()	Smaller subheader	st.subheader('Details')

st.write()	Versatile output (text, data, charts)	st.write('Hello!')
st.text()	Fixed-width plain text	st.text('Code output')
st.markdown()	Render Markdown formatting	st.markdown('**bold**')
st.code()	Syntax-highlighted code	st.code('x = 1', 'python')
st.metric()	KPI metric with delta	st.metric('Accuracy', '94%')
st.dataframe()	Interactive DataFrame table	st.dataframe(df)
st.json()	Formatted JSON display	st.json({'key': 'val'})

3.3 Input Widgets

Widgets let users interact with your app. Each widget returns its current value which you can use in your Python logic:

```
import streamlit as st

# Text input
name = st.text_input('Your name', placeholder='Enter name here')

# Number input
age = st.number_input('Your age', min_value=0, max_value=120, value=25)

# Slider
confidence = st.slider('Confidence threshold', 0.0, 1.0, 0.5)

# Selectbox (dropdown)
model_type = st.selectbox('Choose model', ['Linear', 'Random Forest', 'XGBoost'])

# Multi-select
features = st.multiselect('Select features', ['Age', 'Income', 'Score'])

# Checkbox
show_details = st.checkbox('Show detailed output')

# Button
if st.button('Run Prediction'):
    st.write('Running...')

# File uploader
uploaded = st.file_uploader('Upload CSV', type=['csv'])
```

3.4 Status & Feedback Messages

Streamlit provides styled message boxes to give feedback to the user:

```
st.success('Model loaded successfully!')          # Green
st.info('Please upload a CSV file to begin.')    # Blue
st.warning('Missing values detected in data.')   # Yellow
st.error('An error occurred. Check your input.') # Red

# Progress bar for long operations
import time
```

```
bar = st.progress(0)
for i in range(100):
    time.sleep(0.01)
    bar.progress(i + 1)
```

3.5 Session State

Session state allows you to persist values across reruns. Without it, every widget interaction resets all variables to their initial values:

```
import streamlit as st

# Initialize session state variable
if 'count' not in st.session_state:
    st.session_state.count = 0

if st.button('Click me!'):
    st.session_state.count += 1

st.write(f'Button clicked {st.session_state.count} times')
```

> **When to use st.session_state**
> Use session state whenever you need to remember something between interactions — such as a trained model, a user's login status, a form submission, or step progress in a multi-step workflow.

4. Layout & Organization

A well-structured layout makes your app professional and easy to use. Streamlit provides several layout tools.

4.1 Sidebar

The sidebar is perfect for controls and filters that apply to the whole app. Users can collapse it on smaller screens:

```
import streamlit as st

# Everything inside 'with st.sidebar:' appears in the sidebar
with st.sidebar:
    st.header('Settings')
    model = st.selectbox('Model', ['Logistic Regression', 'Random Forest'])
    n_estimators = st.slider('Trees (if RF)', 10, 200, 100)
    st.markdown('---')  # Horizontal divider
    st.info('Adjust settings above and re-run.')

# Main content area
st.title('ML Prediction App')
st.write(f'Using model: {model}')
```

4.2 Columns

Columns allow you to place content side by side, which is great for comparing metrics or showing inputs and outputs next to each other:

```
col1, col2, col3 = st.columns(3)

with col1:
    st.metric('Accuracy', '94.2%', '+1.3%')

with col2:
    st.metric('Precision', '91.8%', '-0.4%')

with col3:
    st.metric('Recall', '96.1%', '+2.1%')

# Unequal column widths
left, right = st.columns([2, 1])  # left is twice as wide
```

4.3 Tabs & Expanders

```
# Tabs for organizing sections
tab1, tab2, tab3 = st.tabs(['Predictions', 'Model Info', 'Data'])

with tab1:
    st.write('Prediction results go here')

with tab2:
    st.write('Model accuracy, parameters, etc.')

with tab3:
    st.write('Show the raw dataset here')

# Expander for hiding extra content
with st.expander('Show advanced settings'):
    threshold = st.slider('Decision threshold', 0.0, 1.0, 0.5)
    normalize = st.checkbox('Normalize features')
```

5. Caching for Performance

Loading data and models on every rerun would make your app very slow. Streamlit's caching decorators solve this problem elegantly.

5.1 @st.cache_data

Use this decorator for functions that load or process data. The result is cached based on the function's inputs:

```
import streamlit as st
import pandas as pd
```

```
@st.cache_data
def load_data(filepath):
    """This function runs once, then the result is cached."""
    df = pd.read_csv(filepath)
    df.dropna(inplace=True)
    return df

# On subsequent reruns, cached result is returned instantly
df = load_data('data/iris.csv')
st.dataframe(df.head())
```

5.2 @st.cache_resource

Use this decorator for loading ML models, database connections, or any expensive-to-create resource that should be shared across all users:

```
import streamlit as st
import joblib

@st.cache_resource
def load_model():
    """Load model once; share across all users and sessions."""
    model = joblib.load('models/classifier.pkl')
    return model

model = load_model()
st.success('Model loaded from cache!')
```

@st.cache_data	@st.cache_resource
For data (DataFrames, arrays, dicts)	For models, DB connections, APIs
Each user gets their own copy	One shared instance for all users
Serialized and stored to disk	Kept in memory
Use for: load_csv(), preprocess()	Use for: load_model(), get_db()

6. Data Visualization

Streamlit supports multiple charting libraries out of the box. Here are the most common ones for ML projects.

6.1 Built-in Charts

```
import streamlit as st
import pandas as pd
import numpy as np

# Simple line chart
chart_data = pd.DataFrame(np.random.randn(20, 3), columns=['A', 'B', 'C'])
st.line_chart(chart_data)
```

```
# Bar chart
st.bar_chart(chart_data)

# Area chart
st.area_chart(chart_data)
```

6.2 Plotly Charts (Recommended for ML)

Plotly provides interactive, publication-quality charts and is the recommended choice for ML dashboards:

```
import streamlit as st
import plotly.express as px
import pandas as pd

@st.cache_data
def load_iris():
    return px.data.iris()

df = load_iris()

# Scatter plot
fig = px.scatter(
    df, x='sepal_width', y='sepal_length',
    color='species', size='petal_length',
    title='Iris Dataset - Feature Relationships'
)
st.plotly_chart(fig, use_container_width=True)

# Histogram
fig2 = px.histogram(df, x='sepal_length', color='species', barmode='overlay')
st.plotly_chart(fig2, use_container_width=True)
```

6.3 Confusion Matrix with Matplotlib

```
import streamlit as st
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

def plot_confusion_matrix(y_true, y_pred, labels):
    cm = confusion_matrix(y_true, y_pred)
    fig, ax = plt.subplots(figsize=(6, 5))
    disp = ConfusionMatrixDisplay(cm, display_labels=labels)
    disp.plot(ax=ax, cmap='Blues', colorbar=False)
    ax.set_title('Confusion Matrix')
    return fig

fig = plot_confusion_matrix(y_test, y_pred, class_names)
st.pyplot(fig)
```

7. Building a Complete ML Project

Now let us put everything together and build a real, end-to-end machine learning application. We will build an Iris Flower Classifier — a classic beginner ML project — and wrap it in a professional Streamlit interface.

7.1 Project Structure

```
my-streamlit-ml-app/
├── app.py              # Main Streamlit application
├── train_model.py      # Script to train and save the ML model
├── models/
│   └── iris_model.pkl  # Saved trained model
├── data/
│   └── iris.csv        # Dataset (optional)
├── requirements.txt    # Python dependencies
└── README.md           # Project description
```

7.2 Step 1 — Train and Save the Model

Create a file called train_model.py and run it once to generate your saved model:

```
# train_model.py
import os
import joblib
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Evaluate
accuracy = accuracy_score(y_test, model.predict(X_test))
print(f'Test Accuracy: {accuracy:.4f}')

# Save model
os.makedirs('models', exist_ok=True)
joblib.dump(model, 'models/iris_model.pkl')
print('Model saved to models/iris_model.pkl')
```

Run it in the terminal:

```
python train_model.py
# Test Accuracy: 1.0000
# Model saved to models/iris_model.pkl
```

7.3 Step 2 — Build the Streamlit App

Now create app.py with the full interactive application:

```
# app.py — Iris Flower Classifier
import streamlit as st
import numpy as np
import pandas as pd
import joblib
import plotly.express as px
from sklearn.datasets import load_iris

# ── Page Configuration ──────────────────────────────────────
st.set_page_config(
    page_title='Iris Classifier',
    page_icon='🌸',
    layout='wide',
    initial_sidebar_state='expanded'
)

# ── Load Model & Data (cached) ──────────────────────────────
@st.cache_resource
def load_model():
    return joblib.load('models/iris_model.pkl')

@st.cache_data
def load_data():
    iris = load_iris()
    df = pd.DataFrame(iris.data, columns=iris.feature_names)
    df['species'] = [iris.target_names[i] for i in iris.target]
    return df, iris.target_names

model = load_model()
df, class_names = load_data()

# ── Sidebar: User Inputs ────────────────────────────────────
with st.sidebar:

st.image('https://upload.wikimedia.org/wikipedia/commons/4/41/Iris_versicolor_3.jp
g', width=200)
    st.header('Input Features')
    st.markdown('Adjust the flower measurements below:')

    sepal_length = st.slider('Sepal Length (cm)', 4.0, 8.0, 5.4)
    sepal_width  = st.slider('Sepal Width (cm)',  2.0, 4.5, 3.4)
    petal_length = st.slider('Petal Length (cm)', 1.0, 7.0, 1.3)
    petal_width  = st.slider('Petal Width (cm)',  0.1, 2.5, 0.2)

    predict_btn = st.button('Predict Species', type='primary',
use_container_width=True)

# ── Main Area ───────────────────────────────────────────────
st.title('🌸 Iris Flower Species Classifier')
st.markdown('A machine learning app built with **Streamlit** and **scikit-
learn**.')
st.divider()
```

```
# ── Metrics Row ──────────────────────────────────────────────
col1, col2, col3, col4 = st.columns(4)
col1.metric('Sepal Length', f'{sepal_length} cm')
col2.metric('Sepal Width',  f'{sepal_width} cm')
col3.metric('Petal Length', f'{petal_length} cm')
col4.metric('Petal Width',  f'{petal_width} cm')

st.divider()

# ── Prediction ───────────────────────────────────────────────
if predict_btn:
    features = np.array([[sepal_length, sepal_width, petal_length, petal_width]])
    prediction = model.predict(features)[0]
    probabilities = model.predict_proba(features)[0]
    species = class_names[prediction]

    st.subheader('Prediction Result')
    pcol1, pcol2 = st.columns([1, 2])

    with pcol1:
        st.success(f'Predicted Species: **{species.capitalize()}**')
        st.metric('Confidence', f'{probabilities[prediction]*100:.1f}%')

    with pcol2:
        prob_df = pd.DataFrame({
            'Species': class_names,
            'Probability': probabilities
        })
        fig = px.bar(prob_df, x='Species', y='Probability',
                     color='Species', title='Prediction Confidence',
                     color_discrete_sequence=px.colors.qualitative.Set2)
        st.plotly_chart(fig, use_container_width=True)

# ── Data Explorer ────────────────────────────────────────────
st.divider()
tab1, tab2 = st.tabs(['Dataset Explorer', 'Feature Visualization'])

with tab1:
    st.dataframe(df, use_container_width=True)

with tab2:
    x_axis = st.selectbox('X Axis', df.columns[:-1])
    y_axis = st.selectbox('Y Axis', df.columns[:-1], index=1)
    scatter = px.scatter(df, x=x_axis, y=y_axis, color='species',
                         title=f'{x_axis} vs {y_axis}')
    st.plotly_chart(scatter, use_container_width=True)
```

Run your app

```
streamlit run app.py
```

> Your app is now live at http://localhost:8501 — move the sliders in the sidebar and click 'Predict Species' to see real-time predictions from your trained Random Forest model!

8. Forms & File Upload

8.1 Using st.form

Forms group widgets together and only trigger a rerun when the user submits — not on every individual change. This is useful for prediction interfaces where the user fills in many fields before submitting:

```
with st.form('prediction_form'):
    st.subheader('Enter Patient Data')
    age = st.number_input('Age', 0, 120)
    gender = st.selectbox('Gender', ['Male', 'Female', 'Other'])
    bmi = st.number_input('BMI', 10.0, 60.0, 25.0)
    submitted = st.form_submit_button('Predict')

if submitted:
    st.write(f'Running prediction for age={age}, BMI={bmi}')
```

8.2 CSV File Upload

Allow users to upload their own dataset for batch predictions:

```
uploaded_file = st.file_uploader('Upload your CSV dataset', type=['csv'])

if uploaded_file is not None:
    df = pd.read_csv(uploaded_file)
    st.success(f'Loaded {len(df)} rows and {len(df.columns)} columns.')
    st.dataframe(df.head(10))

    if st.button('Run Batch Predictions'):
        predictions = model.predict(df.values)
        df['Prediction'] = predictions
        st.dataframe(df)

        # Download results
        csv = df.to_csv(index=False).encode('utf-8')
        st.download_button('Download Results', csv, 'results.csv', 'text/csv')
else:
    st.info('Please upload a CSV file to begin batch prediction.')
```

9. Deploying Your App

Streamlit Community Cloud is the easiest way to deploy your app for free. Anyone with the link can use it — no server setup required.

9.1 Prepare requirements.txt

Create a requirements.txt file listing all your Python dependencies:

```
streamlit>=1.28.0
scikit-learn>=1.3.0
pandas>=2.0.0
numpy>=1.24.0
plotly>=5.17.0
joblib>=1.3.0
```

9.2 Push to GitHub

```
git init
git add .
git commit -m 'Initial Streamlit ML app'
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
git push -u origin main
```

9.3 Deploy on Streamlit Community Cloud

1. Go to https://share.streamlit.io and sign in with GitHub
2. Click 'New app' and select your repository
3. Set the main file path to app.py
4. Click 'Deploy!' and wait 1-3 minutes
5. Your app is now live at a public URL you can share

> **Free Forever**
> Streamlit Community Cloud is completely free for public repositories. You get unlimited public apps with no time limits. Private repositories require a paid plan.

9.4 Common Deployment Issues

Issue	Cause	Fix
ModuleNotFoundError	Package not in requirements.txt	Add the missing package to requirements.txt
FileNotFoundError for model	Model file not in GitHub repo	Commit the models/ folder or use st.cache to retrain
App crashes on startup	Import error or syntax issue	Check logs in Streamlit Cloud dashboard
Slow loading	No caching used	Add @st.cache_data and @st.cache_resource

10. Best Practices & Tips

10.1 Performance

- Always cache data loading and model loading with @st.cache_data and @st.cache_resource
- Use st.spinner() for long operations to show a loading indicator
- Avoid loading large files on every rerun — cache everything heavy
- Use st.empty() and placeholder patterns to update parts of the UI efficiently

10.2 User Experience

- Always use st.set_page_config() at the very top of your script
- Provide default values for all widgets so the app works on first load
- Use st.sidebar for controls and the main area for results
- Add st.info() messages to guide users who haven't interacted yet
- Use st.divider() or st.markdown('---') to separate sections visually

10.3 Code Organization

```
# Recommended structure for a production Streamlit app

import streamlit as st

# 1. Page config — must be FIRST st. call
st.set_page_config(page_title='My App', layout='wide')

# 2. Load resources (with caching)
@st.cache_resource
def load_model(): ...

@st.cache_data
def load_data(): ...

# 3. Define helper functions
def preprocess(input_data): ...
def postprocess(prediction): ...

# 4. Sidebar inputs
with st.sidebar:
    ...

# 5. Main content area
st.title('...')
# ... display, logic, charts ...
```

10.4 Common Mistakes to Avoid

- Do not put st.set_page_config() anywhere except the very first line of your script
- Do not use global variables to store state — use st.session_state instead
- Do not load heavy files inside functions that are not cached
- Do not forget to handle the case where a file uploader has no file yet (check for None)

- Do not hardcode file paths — use os.path.join() for cross-platform compatibility

11. Next Steps & Resources

Congratulations on completing this guide! You now have the skills to build and deploy a real ML application with Streamlit. Here is what to explore next:

11.1 Advanced Streamlit Features

- Multi-page apps — organize large apps across multiple pages
- Custom components — embed React components in Streamlit
- Streamlit Authenticator — add login/password protection
- st.experimental_connection — connect to databases and APIs
- Theming — customize colors and fonts in .streamlit/config.toml

11.2 Project Ideas to Build

Project	ML Technique	Difficulty
House Price Predictor	Linear/Ridge Regression	Beginner
Sentiment Analyzer	Text Classification (NLP)	Beginner
Customer Churn Dashboard	Logistic Regression	Beginner
Image Classifier (MNIST)	CNN (TensorFlow/PyTorch)	Intermediate
Stock Price Forecaster	LSTM / ARIMA	Intermediate
Real-time Object Detection	YOLO / OpenCV	Advanced
Recommendation Engine	Collaborative Filtering	Advanced

11.3 Useful Resources

- Streamlit Documentation: https://docs.streamlit.io
- Streamlit Component Gallery: https://streamlit.io/components
- Streamlit Community Forum: https://discuss.streamlit.io
- Awesome Streamlit (GitHub): github.com/MarcSkovMadsen/awesome-streamlit
- Scikit-learn Documentation: https://scikit-learn.org/stable/

You're Ready to Build!

Start small, build consistently, and ship your first app today.